Runtime and Memory Scaling of the Basic Triple-Loop Matrix Multiplication in C, Java and Python

BIG DATA · Individual Assignment 1 · Grado en Ciencia e Ingeniería de Datos, ULPGC · 2026-2027

Jose Jesus Perez Suarez

October 2026

Abstract

We compare the same straightforward i-j-k dense matrix multiplication, written in C, Java and Python, on one documented 2-vCPU cloud virtual machine, and ask how the matrix side length n affects runtime and peak memory. All three programs read identical binary input files, use IEEE-754 binary64 arithmetic in the same order, and produce bit-identical results. Timing covers the multiplication kernel only, with warm-up runs and five measured repetitions per configuration. Python was 33–95× slower than C at every size it completed and exceeded the 60 s budget at n = 1000. C and Java were within a factor of about two of each other, but their ranking changed with n: neither language was uniformly faster. Runtime did not follow a single cubic curve: for C, sizes that are multiples of 512 were 3–10× slower per operation than neighbouring sizes. A supplementary experiment that shifts each row by 64 bytes removed most of that penalty (4.2× faster at n = 512, 7.3× at n = 1024, 8.9× at n = 1536), which supports a cache-conflict explanation tied to the storage layout rather than to the language. Peak memory grew by about 8, 20 and 40 bytes per matrix element for C, Java and Python. These conclusions are limited to the tested implementations, settings and machine.

1 Introduction and objective

Matrix multiplication is the reference kernel of dense linear algebra and a standard vehicle for studying how algorithms interact with hardware. The textbook algorithm performs $2n^3$ floating-point operations, so its runtime is expected to grow cubically in n. In practice the constant in front of n^3 depends on how the language executes the inner loop and on how the data layout interacts with the memory hierarchy.

Research question. *How does matrix size n affect the runtime and peak memory of the same basic triple-loop matrix multiplication implemented in C, Java and Python, under fixed and documented execution conditions?*

Hypotheses (stated before measuring).

- **H1** – For each implementation the runtime grows approximately as n^3.
- **H2** – C and Java (after JIT warm-up) will be within a small factor of each other, and CPython will be one to two orders of magnitude slower, because each inner-loop iteration is interpreted and allocates boxed floating-point objects.
- **H3** – Peak memory grows as a constant times n^2: about 8 bytes per element for C and Java, more for Python (an 8-byte pointer plus a 24-byte float object), on top of a fixed runtime footprint that is largest for the JVM.

2 Background

For n × n matrices the classical algorithm computes $C_{ij} = \Sigma_k A_{ik} B_{kj}$, with n^3 multiply-add pairs [1]. In the i-j-k order used here, the innermost loop reads one row of A contiguously but walks down a column of B with a stride of one row. On a row-major layout that stride is 8n bytes, so every access to B touches a different cache line, and reuse of each 64-byte line of B depends on that line surviving in cache while j advances [2], [3]. When the stride is a large power of two, all elements of a column map to a small number of cache sets and evict each other (self-interference or conflict misses) [4]. This effect is well known to depend on the leading dimension of the stored matrix, not on the arithmetic.

Benchmarking managed runtimes requires warm-up because the JVM interprets and then JIT-compiles hot methods, and results should be reported with their variability rather than as single numbers [5], [6]. Since Java 17 all floating-point arithmetic is strict IEEE-754 [7], and GCC can be told not to fuse multiply and add (`-ffp-contract=off`), which lets all three implementations produce identical rounding.

3 Methodology

3.1 Implementations

All code is in the repository (`src/`). Each implementation contains the same kernel:

```
for i in 0..n-1:
    for j in 0..n-1:
        s = 0.0
        for k in 0..n-1:
            s += A[i][k] * B[k][j]
        C[i][j] = s
```

Table 1: The three implementations.

	C	Java	Python
Storage	one contiguous row-major `double*` block, index `i*n+k`	`double[][]`: n separate row arrays (each contiguous, 16-byte object header)	list of n lists of pointers to boxed `float` objects
Numeric type	binary64	binary64	binary64 (CPython `float`)
Build / runtime	GCC 13.3.0, `-O2 -std=c11 -ffp-contract=off`	OpenJDK 21.0.12, `-Xmx2g -XX:+UseSerialGC`	CPython 3.13.16
Timer	`clock_gettime`, `CLOCK_MONOTONIC`	`System.nanoTime()`	`time.perf_counter_ns()`
Warm-ups	1	5 (JIT)	1

No library multiplication routine is used in any implementation. Each storage layout is the idiomatic choice for its language; this difference is unavoidable for Python and is a deliberate, documented difference for Java (see Discussion). NumPy is used only to generate inputs and as the correctness reference.

3.2 Inputs

All languages read the same files: $n \times n$ little-endian binary64 values, row-major, generated by `scripts/generate_inputs.py` with NumPy's PCG64 generator seeded by `SeedSequence([20261014, n])`; values are uniform on $[-1, 1)$. SHA-256 hashes of every file are stored in `data/manifest.json`. The declared size sequence was 64, 128, 256, 384, 512, 768, 1000, 1024, 1536 and 2048. The value 1000 was added before the campaign, next to 1024, to separate the effect of n from the effect of n being a power of two.

3.3 Environment

Table 2: Execution environment.

Item	Value
CPU	Intel Xeon Processor @ 2.10 GHz (KVM guest), 2 physical cores, 2 logical processors, 1 thread/core
Caches (per core)	L1d 48 KiB 12-way; L2 2 MiB 16-way (2048 sets, 64-byte lines); L3 260 MiB shared with the host
RAM	8.4 GB, no swap

Item	Value
OS	Ubuntu 24.04.5 LTS, Linux 6.18; transparent huge pages = `madvise`
Threads	C and Python: 1. Java: 1 application thread; the JVM also runs 2 JIT-compiler threads; Serial GC (single-threaded)

The full record is in `results/environment.json`. The machine is a shared cloud VM, so interference from other tenants cannot be excluded; it is visible as variability in the results.

3.4 Measurement protocol

- **Timer boundary: multiplication kernel only.** Inputs are loaded and A, B and C are allocated before timing. The kernel overwrites every C_{ij}, so no re-initialisation is needed. Process start-up, JVM start-up and file loading are not timed.
- Each (language, n) configuration runs in a new process: warm-ups (not reported), then 5 measured repetitions. Kernels shorter than 50 ms are repeated in a batch and the reported value is time per multiplication (batch size is recorded; e.g. 410 for C at n = 64).
- After each repetition the program prints a checksum of C, so the computation cannot be removed by the compiler. Checksums agreed numerically across languages for every n.
- The harness runs sizes in increasing order and, for each size, C → Java → Python.
- **Budgets, declared in advance.** 60 s per multiplication and 4 GB of estimated array storage. If one warm-up multiplication exceeds 60 s, the configuration is recorded as *over budget* and larger sizes are not attempted for that language. No time is assigned to such runs.
- **Memory metric.** Peak resident set size (VmHWM from `/proc/self/status`), read by each program before exiting. It includes the runtime (JVM, interpreter) and the input-loading phase, not only the matrices. We do not use `getrusage`'s `ru_maxrss` from the parent: on Linux it is inherited across `execve` and reported about 11 MB for a trivial C program spawned from the Python harness, whose own VmHWM was 1.7 MB.
- Summary statistics: median of the 5 repetitions; variability as the interquartile range (IQR) and min–max. All 120 raw repetitions are in `results/raw_times.csv`; none were excluded.

3.5 Correctness

`scripts/validate.py` was run before benchmarking and passed 34 of 34 checks for all three programs: (i) a hand-computed 2×2 product, exact; (ii) $A \cdot I = I \cdot A = A$ for n = 17, exact; (iii) $A \cdot 0 = 0$ for n = 33, exact; (iv) random matrices with n = 1, 3, 64, 100, 129 compared with NumPy; (v) bitwise equality of the three implementations on case (iv).

For (iv) the tolerance is the classical forward error bound for a dot product computed by recursive summation, $|\mathrm{fl}(C) - C| \le \gamma_n (|A||B|)$ with $\gamma_n = nu / (1 - nu)$ and $u = 2^{-53}$ [1]. Because the reference itself is rounded, we accept $|C - C_{\mathrm{NumPy}}| \le 2\gamma_n (|A||B|)$ element-wise. This tolerance grows with *n* and with the magnitudes involved, which a fixed relative tolerance would not. The largest observed error was 7.1×10^{-15} at n = 129, against a bound of 1.2×10^{-12}. All three implementations agreed bit for bit, as expected from identical summation order and the absence of fused multiply-add.

4 Results

4.1 Runtime

Table 3 gives the median time per multiplication with the relative IQR in parentheses, and the ratio to C. Figure 1 shows the same medians on log-log axes.

Table 3: Median time per multiplication (relative IQR), 5 repetitions per cell.

n	C	Java	Python	Java / C	Python / C
64	115 µs (0.3%)	216 µs (4.8%)	10.9 ms (2.0%)	1.87	94.9
128	2.2 ms (5.8%)	1.3 ms (1.1%)	85.5 ms (0.3%)	0.56	38.2
256	16.3 ms (0.8%)	13.0 ms (47.1%)	987.2 ms (9.9%)	0.79	60.4
384	57.7 ms (0.7%)	68.9 ms (7.7%)	5.21 s (7.8%)	1.19	90.3
512	448.8 ms (6.6%)	185.7 ms (2.6%)	14.93 s (2.5%)	0.41	33.3
768	721.7 ms (8.5%)	822.4 ms (4.6%)	45.49 s (1.9%)	1.14	63.0
1000	1.07 s (9.6%)	2.04 s (4.8%)	over budget	1.91	–
1024	10.03 s (1.8%)	1.95 s (6.6%)	not run	0.19	–
1536	38.10 s (1.6%)	31.32 s (3.9%)	not run	0.82	–
2048	over budget	over budget	not run	–	–

Over budget: one warm-up multiplication took 127.4 s (Python, n = 1000), 95.1 s (C, n = 2048) and 96.0 s (Java, n = 2048), above the 60 s limit. *Not run:* not attempted after the language exceeded the budget at a smaller size. **Largest tested size within budget:** Python 768; C 1536; Java 1536. **Largest size attempted:** Python 1000; C and Java 2048.

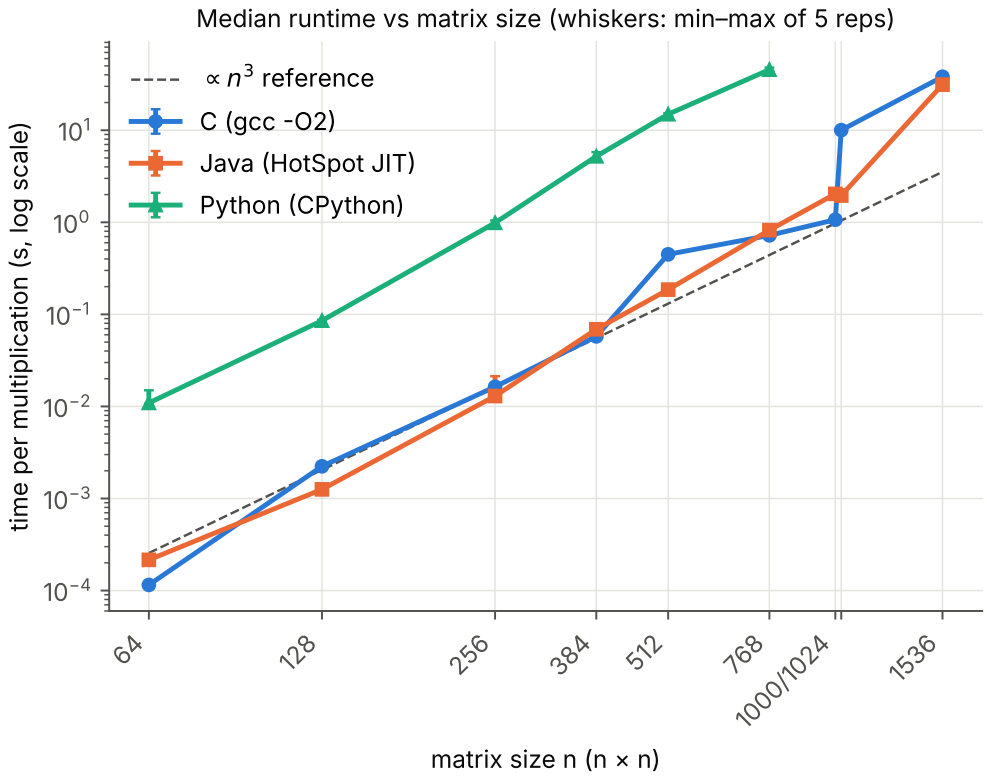


Figure 1: Median runtime per multiplication against n (log-log). Whiskers span the minimum and maximum of the 5 repetitions; the dashed line is n^3 anchored at C's n = 256 median.

Variability was small in most cells (relative IQR below 10%). The exception is Java at n = 256 (IQR 47%): two of the five repetitions took about 21 ms instead of 13 ms. All five are kept; we did not identify the cause (possibly a late JIT recompilation or a GC pause).

4.2 Growth with n

Figure 2 divides each median by n^3, giving the cost of one inner-loop iteration. A pure cubic algorithm on an ideal machine would give flat lines. Table 4 lists least-squares slopes of log(time) against log(n) on three subsets declared in `scripts/analyze.py`.

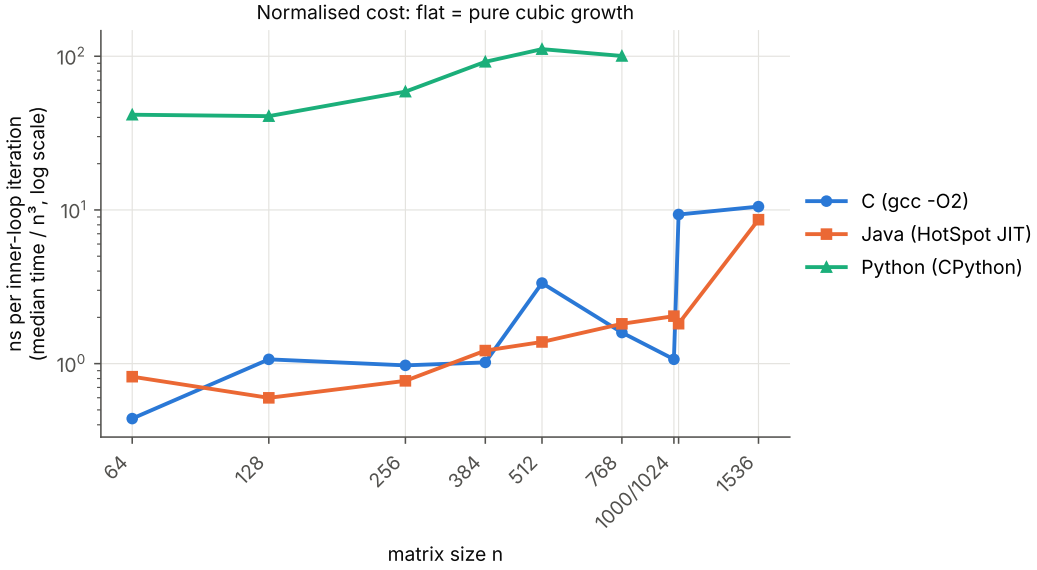


Figure 2: Median time divided by n^3 (nanoseconds per inner-loop iteration).

Table 4: Fitted exponent b in time $\propto n^b$.

Subset	C	Java	Python
$128 \le n \le 384$	2.95	3.61	3.72
all completed $n \ge 256$	4.14	4.04	3.50 (256–768)
$n \ge 256$ excluding multiples of 512	3.20	3.69	3.45 (256, 384, 768)

4.3 Supplementary diagnostic: row padding in C

To test whether the slow C sizes come from the layout, a separate C program (`src/c/diag_leading_dim.c`, not part of the language comparison) uses the same kernel with a leading dimension `ld = n + 8`, i.e. each row starts 64 bytes later than in the unpadded layout. The arithmetic is unchanged. One warm-up and 3 measured repetitions were run per cell.

Table 5: C kernel, unpadded (`ld = n`) vs padded by one cache line (`ld = n + 8`), median of 3.

n	ld = n	ld = n + 8	Speed-up
512	420.8 ms	100.2 ms	4.20
768	758.4 ms	485.4 ms	1.56
1000	912.7 ms	1.31 s	0.69
1024	10.01 s	1.37 s	7.32
1536	38.59 s	4.34 s	8.88

The unpadded times reproduce the main campaign within 10% (e.g. 10.01 s vs 10.03 s at n = 1024).

4.4 Memory

Table 6: Peak resident memory (MB) of the benchmark process. † = process stopped by the time budget after one multiplication.

n	C	Java	Python	$3 \times 8n^2$ (raw doubles)
64	2.3	43.3	10.9	0.1
128	2.6	44.0	12.4	0.4
256	3.8	46.6	18.3	1.6
384	5.8	50.5	28.4	3.5
512	8.5	55.8	42.2	6.3
768	16.4	71.5	81.8	14.2
1000	26.2	93.0	(131.1)†	24.0
1024	27.3	105.3	not run	25.2
1536	58.8	181.3	not run	56.6
2048	(102.8)†	(225.7)†	not run	100.7

A linear fit of peak RSS against the number of stored elements $3n^2$ (completed runs, $n \geq 256$) gives a marginal cost of **8.0 bytes per element for C, 19.7 for Java and 40.3 for Python**, with intercepts (fixed footprint) of 2.2, 40.1 and 10.5 MB.

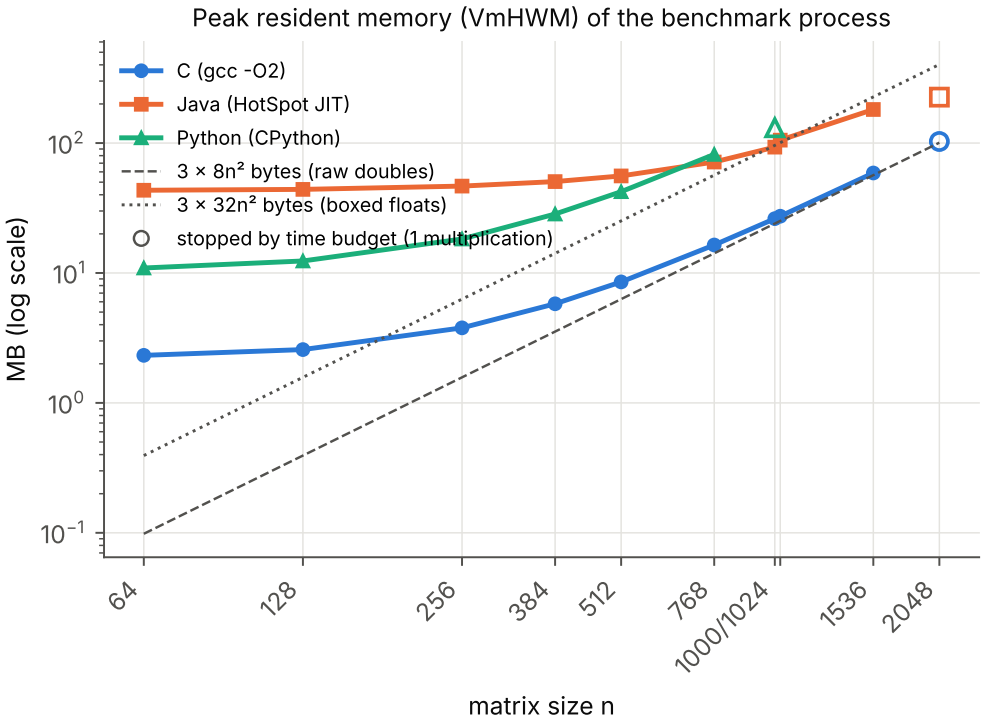


Figure 3: Peak resident memory against n. Hollow markers: processes stopped by the time budget. Dashed and dotted lines: storage for $3n^2$ raw doubles and for $3n^2$ boxed Python floats.

5 Discussion

H1 – cubic growth holds only within a stable memory regime. In the small, cache-friendly range ($128 \leq n \leq 384$) C grows with exponent 2.95, very close to 3. Over the whole range the fitted exponent is above 4 for C and Java, because the cost per iteration (Figure 2) is not constant: it increases once the working set of the column walk no longer fits in cache. The cubic model therefore describes the *work* correctly, but the constant in front of n^3 depends on n through the memory hierarchy. For

Python the exponent is about 3.5 because its per-iteration cost rose from about 41 ns ($n \le 128$) to about 100–110 ns ($n \ge 512$); we did not measure the cause.

The power-of-two anomaly is a layout effect. C took 10.03 s at n = 1024 but only 1.07 s at n = 1000, although n = 1024 has only 7% more work. The same pattern appears at n = 512 and n = 1536 (both multiples of 512). With a stride of 8n bytes, a multiple of 512 gives a stride that is a multiple of 4096 bytes, so every element of a B column has the same offset within its 4 KiB page. On this CPU the L1 and L2 set index uses address bits 6–11 (fixed by the page offset) and, for the L2, bits 12–16, which depend on the physical pages chosen by the OS because transparent huge pages were not applied. A whole column can then occupy at most 32 of the 2048 L2 sets, i.e. $32 \times 16 = 512$ lines, while the i-j-k loop needs about n lines (one per row of B) to stay in cache to reuse each line for 8 consecutive values of j. For n = 1024 and 1536 that capacity is clearly too small, so each B access is served from L3 or memory. This is an explanation consistent with the cache geometry, not a direct measurement (hardware counters were not available in the VM). Table 5 is the direct test: shifting every row by one cache line changes no arithmetic yet made n = 1024 7.3× faster and n = 1536 8.9× faster, close to the cost per iteration of neighbouring sizes. At n = 1000, where the stride is not a multiple of 4096, padding brought no gain (0.69×, i.e. slower), which we cannot explain from these data.

H2 – partially supported, and the C/Java ranking is not stable. Python was 33–95× slower than C at every completed size, consistent with interpreting each iteration: the statement `s += A[i][k] * B[k][j]` performs two nested list indexings per operand and creates two new float objects per iteration. C and Java were within a factor of about two at most sizes, but Java was faster at n = 128, 256, 512, 1024 and 1536 and slower at 64, 384, 768 and 1000. The largest Java "advantage" (5.1× at n = 1024) is very likely the same layout effect seen in Table 5: in `double[][]` each row is a separate object with a 16-byte header, so consecutive rows are not 4096 bytes apart and the column walk is not confined to a few cache sets. That is a property of the storage layout, not of the Java language or the JIT. Java at n = 1536 (31.3 s) was however still slow, unlike padded C (4.3 s); possible causes include the placement of row objects on the heap, the extra load of the row pointer and array length for bounds checks, and TLB reach, but we did not test them. Java's slower result at n = 64 (1.87× C) may reflect warm-up being insufficient for such short kernels or code differences after JIT; we did not inspect the generated machine code.

H3 – supported, with one correction. C used 8.0 bytes per element over a 2.2 MB baseline, matching three double arrays. Python used 40.3 bytes per element: the predicted 32 (pointer + 24-byte float object) plus list overhead and the temporary `array('d')` buffer used during loading. Java used 19.7 bytes per element rather than the predicted 8, because peak RSS includes the input-loading phase: `Files.readAllBytes` creates a byte array of $8n^2$ bytes per matrix that becomes garbage, and the JVM keeps heap pages it has touched. The JVM also has the largest fixed footprint (about 40 MB). The reported Java heap in use after the run (from `Runtime.totalMemory() - freeMemory()`) was 60 MB at n = 1536, against 57 MB of raw arrays, which supports this interpretation. For all three languages, memory was never the limiting factor on this machine: runtime budgets were reached long before the 4 GB memory budget.

Limitations. (i) One shared cloud VM with 2 vCPUs; results may differ on other hardware, and we cannot exclude interference from other tenants. (ii) Storage layouts differ by language, so the C/Java comparison mixes language and layout effects (Table 5 shows the layout effect can be up to 9×). (iii) Five repetitions per cell (three in the diagnostic); medians of cells with high IQR, such as Java at n = 256, are less certain. (iv) Peak RSS includes runtime and loading costs, so it is a process metric, not a measure of the kernel's data. (v) Each configuration was run in one process in a fixed order; order effects were not randomised. (vi) No hardware performance counters were available, so cache explanations are inferred from timing.

6 Conclusions

On the tested machine and settings: (1) the basic triple loop in CPython was 33–95× slower than in C and could not complete a 1000×1000 multiplication within 60 s; (2) C and Java performed within about a factor of two of each other at most sizes, and which one was faster depended on n, so these data do not support a general claim that either language is faster; (3) runtime grew close to n^3 only while the memory access pattern stayed in the same cache regime, and matrix sizes that are multiples of 512 caused 4–9× slowdowns in C that disappeared when the rows were padded by 64 bytes; (4) peak memory grew linearly in n^2 at about 8, 20 and 40 bytes per element for C, Java and Python. Unresolved: why Java remains slow at n = 1536, why padding slowed C at n = 1000, and what raises Python's per-iteration cost at larger n. Hardware-counter measurements (cache and TLB misses) would be the next step, and Assignment 2 can build on the padding result with loop reordering and blocking.

AI use disclosure

Tool. Claude (Anthropic), used through the Claude app with code execution, October 2026.

Purpose and affected parts. The assistant was used well beyond language revision. It wrote the three implementations (`src/`), the input generator and the validation, benchmark and analysis scripts (`scripts/`); it ran the correctness tests and all experiments on a cloud virtual machine (the environment described in Section 3.3, not the author's own computer); it produced the figures; and it drafted the text of this report, including the choice of references. In practice this affects every part of the submission.

Provenance of the numbers. Every number in the tables and figures comes from the raw measurements in `results/` through `scripts/analyze.py`; no value was estimated or typed in by hand. The tables in this report were copied from the tables that script generates (`results/report_tables.md`, `results/tables.md`).

Responsibility. In line with the course policy, the author is responsible for understanding the code, validating the measurements against the raw data, verifying that each reference exists and supports the claim it is cited for, and defending the conclusions.

References

[1] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[2] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74.

[5] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.

[6] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '15)*, 2015, Art. no. 73.

[7] J. D. Darcy, "JEP 306: Restore always-strict floating-point semantics," OpenJDK, 2021. [Online]. Available: https://openjdk.org/jeps/306

Reproducibility materials

Repository: https://github.com/Pepote145/bigdata-assignment01-matmul. It contains the source code, `README.md` with exact build and run commands, `config.json` with every experimental parameter, input generation script and `data/manifest.json` (hashes), all raw measurements (`results/raw_times.csv`, `results/runs.csv`, `results/diag_leading_dim.csv`), the validation log (`results/validation.csv`), the environment record (`results/environment.json`) and the scripts that produce every table and figure (`scripts/analyze.py`). `make all` reproduces the study.